

4×1 MUX and 1×4 DEMUX – Inputs & Outputs								
Circuit	Input	Size	Input MSB	Input LSB	Output	Size	Output MSB	Output
4×1 MUX	I[3:0]	4-bit	I[3]	I[0]	Y	1-bit	—	—
4×1 MUX	S[1:0]	2-bit	S[1]	S[0]	—	—	—	—
1×4 DEMUX	D	1-bit	—	—	Y[3:0]	4-bit	Y[3]	Y[0]
1×4 DEMUX	S[1:0]	2-bit	S[1]	S[0]	—	—	—	—

MUX

Main code

```
module mux_4to1 (
    input wire [3:0] I,
    input wire [1:0] S,
    output reg Y
);
```

```
always @(*) begin
    case (S)
        2'b00: Y = I[0];
        2'b01: Y = I[1];
        2'b10: Y = I[2];
        2'b11: Y = I[3];
        default: Y = 1'b0;
    endcase
end
```

```
endmodule
```

DEMUX

```
`timescale 1ns/1ps
```

```

module mux_4to1_tb;

reg [3:0] I;
reg [1:0] S;
wire Y;

mux_4to1 uut (
    .I(I),
    .S(S),
    .Y(Y)
);

initial begin

    I = 4'b1010; S = 2'b00; #10;
    I = 4'b1010; S = 2'b01; #10;
    I = 4'b1010; S = 2'b10; #10;
    I = 4'b1010; S = 2'b11; #10;

    $finish;
end

initial begin

    $monitor("Time=%0t | I=%b | S=%b | Y=%b",
        $time, I, S, Y);
end

endmodule

```

DEMUX

MAIN CODE

```
module demux_1to4 (  
    input wire D,  
    input wire [1:0] S,  
    output reg [3:0] Y  
);  
  
always @(*) begin  
    Y = 4'b0000;  
  
    case (S)  
        2'b00: Y[0] = D;  
        2'b01: Y[1] = D;  
        2'b10: Y[2] = D;  
        2'b11: Y[3] = D;  
        default: Y = 4'b0000;  
    endcase  
end  
  
endmodule
```

TEST BENCH CODE

```
`timescale 1ns/1ps  
  
module demux_1to4_tb;  
  
    reg D;  
    reg [1:0] S;
```

```
wire [3:0] Y;
```

```
demux_1to4 uut (  
    .D(D),  
    .S(S),  
    .Y(Y)  
);
```

```
initial begin
```

```
    D = 1'b1; S = 2'b00; #10;
```

```
    D = 1'b1; S = 2'b01; #10;
```

```
    D = 1'b1; S = 2'b10; #10;
```

```
    D = 1'b1; S = 2'b11; #10;
```

```
    $finish;
```

```
end
```

```
initial begin
```

```
    $monitor("Time=%0t | D=%b | S=%b | Y=%b",
```

```
        $time, D, S, Y);
```

```
end
```

```
endmodule
```